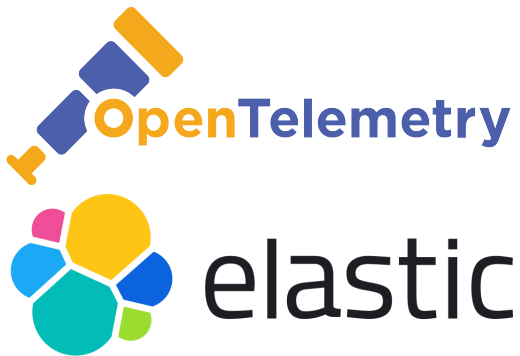


Write safe native code for JVM with Rust

Mateusz Rzeszutek

14th May 2026

Who am I?



I'm an emeritus maintainer of the
[OpenTelemetry Instrumentation for Java](#).

Currently working for Elastic as part of
the Core JVM Infra team.

What we're going to talk about

- Why even use native code in the first place;
- A little bit about the “old” way of using native code, i.e Java Native Interface (*JNI*);
- Then, the Foreign Function and Memory (*FFM*) API, added recently in JDK 22;
- And how to write the native parts in Rust, not just plain C.

Why use native extensions in Java?

- Performance
 - Low level memory control: custom memory layouts, pointer arithmetics
 - Single Instruction Multiple Data (*SIMD*)
 - Predictability, avoiding GC overhead
- Code reuse
 - Native libraries with thin Java API layer
- System-level capabilities
 - Low level system APIs (e.g. Netty and `epoll/kqueue`)

Why NOT use native extensions in Java?

- Complexity: need separate binaries per OS/architecture
- Complexity: C/C++ quirks and undefined behaviors
- Complexity: memory management and ownership
- Debugging is a major pain
- You can crash the JVM easily
- There is an extra overhead when crossing native/JVM boundary, and JVM optimizations won't apply
- And lots of others. . .

The old way: JNI

Java Native Method

```
class JniExample {  
    static native String sayHello(String name);  
}
```

The old way: JNI

Generate C/C++ headers

```
javac -h hello_native/src \  
src/main/java/org/example/JniExample.java
```

Implement C/C++ native method

```
JNIEXPORT jstring JNICALL Java_org_example_JniExample_sayHello(  
    JNIEnv *, jclass, jstring) {  
    // ...  
}
```

The old way: JNI

Set library path

```
java -Djava.library.path=/path/to/cpp/output \  
-jar jniexample.jar
```

Load native library

```
System.loadLibrary("hello_native");
```

→ *IDE*

JNI: the good, the bad and the ugly

Good

- Access to native ecosystems
- Performance (in case of careful usage)
- Low-level control

Bad and ugly

- Unsafe, very easy to shoot yourself in the foot
- The glue code (between JVM and native) is in C/C++
- Java exceptions vs error codes
- Tricky interactions with GC, memory management, ownership
- Lack of JVM optimisations, extra overhead when crossing boundary

Java Foreign Function and Memory API

FFM vs JNI

FFM examples

Passing Java code to Rust: HTTP server

Thank you!

Presentation sources

<https://github.com/mateuszrzeszutek/write-safe-native-code-for-jvm-with-rust>

QR Code



Q&A